Data Science: Simpsons & Pandas

Dr. Konrad Völkel, Sommersemester 2026

Simpson's Paradox

(hat nix mit Pandas zu tun)



"Robotic sheep on Windows XP background, Kelvin timeline"

Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Beispiel (straight from Wikipedia bzw. [1])

Man vergleicht zwei Behandlungen (A, B) für Nierensteine. Behandlung A hat eine Erfolgsrate von 78% (273/350), hingegen Behandlung B hat 83% (289/350) Erfolg.

[1] C. R. Charig; D. R. Webb; S. R. Payne; J. E. Wickham (29 March 1986). "Comparison of treatment of renal calculi by open surgery, percutaneous nephrolithotomy, and extracorporeal shockwave lithotripsy". *Br Med J (Clin Res Ed)*. **292** (6524): 879–882. doi:10.1136/bmj.292.6524.879

Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Beispiel:

Man vergleicht zwei Behandlungen (A, B) für Nierensteine. Behandlung A hat eine Erfolgsrate von 78% (273/350), hingegen Behandlung B hat 83% (289/350) Erfolg.

Ist B die bessere Behandlung?

Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Beispiel:

Man vergleicht zwei Behandlungen (A, B) für Nierensteine. *Es gibt kleine und große Nierensteine.*

Bei *kleinen* Nierensteinen ist die Erfolgswahrscheinlichkeit von A bei 93% (81/87), hingegen B bei 87% (234/270).

Ist A die bessere Behandlung bei kleinen Nierensteinen?

Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Beispiel:

Man vergleicht zwei Behandlungen (A, B) für Nierensteine. *Es gibt kleine und große Nierensteine.*

Bei *großen* Nierensteinen ist die Erfolgswahrscheinlichkeit von A bei 73% (192/263), hingegen B bei 69% (55/80).

Ist A die bessere Behandlung bei großen Nierensteinen?

Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Beispiel:

Man vergleicht zwei Behandlungen (A, B) für Nierensteine. *Es gibt kleine und große Nierensteine.*

	Behandlung A	Behandlung B
Kleine Nierensteine	**93% (81/87)**	87% (234/270)
Große Nierensteine	**73% (192/263)**	69% (55/80)
Alle Nierensteine	78% (273/350)	**83% (289/350)**

Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Beispiel:

Der “Confounder” ist die Größe der Nierensteine.

B kommt häufiger bei kleinen zum Einsatz:

	Behandlung A	Behandlung B
Kleine Nierensteine	**93% (81/87)**	87% (234/270)
Große Nierensteine	**73% (192/263)**	69% (55/80)
Alle Nierensteine	78% (273/350)	**83% (289/350)**

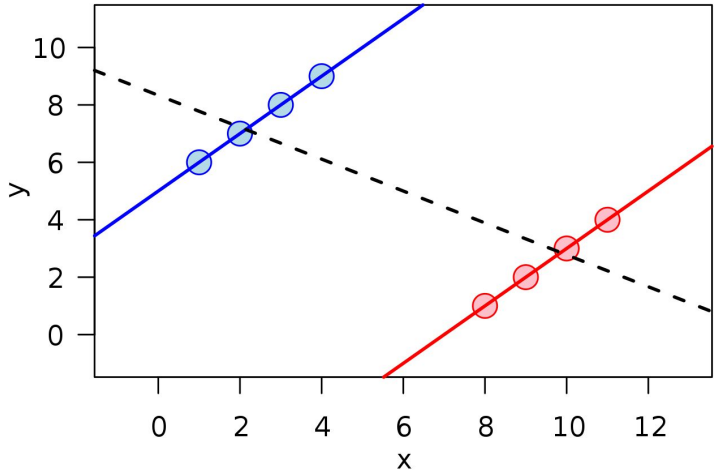
A ist besser und teurer, wird also nur im Ernstfall genutzt.

Simpson’s paradox

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Abstrakter haben wir es oft mit Verteilungen zu tun, die als Mischung (Summe) von Verteilungen auftreten,

deren einzelne Tendenz der Summentendenz zuwider läuft:



Simpson’s paradox

Heinrich Heine
Universität Düsseldorf

Oder: warum Clustern und Verteilungen mischen eine verdammt gute Idee ist

Take-Home-Message:

Die globalen Tendenzen (z.B. Korrelationen) einer Verteilung sind trügerisch, wenn es latente Variablen gibt, wie z.B. die Zugehörigkeit zu Klassen.

Übungsaufgabe: machen Sie sich das am Iris-Datensatz klar.

Pandas

Dataframes



“Meditating Panda, ink”

Pandas

Heinrich Heine
Universität Düsseldorf

Dataframes

Die grundlegende Datenstruktur bei Pandas ist der Pandas `DataFrame`.

- wie ein data.frame in R
- ein bisschen wie ein Numpy record array
- aber: 2D (keine allgemeinen Tensoren), d.h.
- Tabellarische Daten
- “Ein Excel Sheet in Python”

Pandas

Heinrich Heine
Universität Düsseldorf

Dataframes

```
import pandas as pd
columns = { 'colname1':[val1, …], …}
df = pd.DataFrame(columns)
print(df['colname1']) # val1 …
print(df.colname1)    # geht genauso
df.dtypes
# da wird klar, dass Numpy dahinter steckt
```

Pandas

Dataframes

```
import pandas as pd
# Laden (geht auch mit URL):
df = pd.read_csv('data.csv')
…
# Speichern:
df.to_csv('modified-data.csv',
          encoding='utf-8')
```

Pandas

Heinrich Heine
Universität Düsseldorf

Series

Die zweite grundlegende Datenstruktur bei Pandas ist die Pandas `Series`.

Sie verhält sich wie Spalten einer Tabelle,
also eindimensionale Arrays.

```
s = df['colname1'].squeeze()
# type(s) == <class 'pandas.core.series.Series'>
```

Pandas

Dokumentation

Die Community rund um Pandas und Scientific Computing mit Python allgemein hat sehr viel Material produziert:

pandas.pydata.org/docs/getting_started/tutorials.html

die API-Referenz ist ebenfalls exzellent (wie bei Numpy), z.B. ist pandas.DataFrame.groupby sehr nützlich.

Pandas

Heinrich Heine
Universität Düsseldorf

Dokumentation

Ein Punkt, den man einmal gesehen haben muss ist das Typensystem von Pandas. Es erweitert die Typen von Numpy:

pandas.pydata.org/docs/reference/arrays.html

Pandas

Heinrich Heine Universität Düsseldorf

Typensystem

Kind of Data	pandas Data Type	Scalar	Array
TZ-aware datetime	DatetimeTZDtype	Timestamp	Datetime data
Timedeltas	(none)	Timedelta	Timedelta data
Period (time spans)	PeriodDtype	Period	Timespan data
Intervals	IntervalDtype	Interval	Interval data
Nullable Integer	Int64Dtype, …	(none)	Nullable integer
Categorical	CategoricalDtype	(none)	Categorical data
Sparse	SparseDtype	(none)	Sparse data
Strings	StringDtype	str	Text data
Boolean (with NA)	BooleanDtype	bool	Boolean data with missing values

SQL

Joins & Selects



“Flock of happy floppy disks and CD-ROMs flying above beautiful landscape”

Pandas

Heinrich Heine
Universität Düsseldorf

Mit Pandas DataFrames kann man alle Operationen durchführen, die man auch in SQL ausdrücken kann.

Wir lernen, wie SQL Queries zu Pandas Code korrespondieren.

SQL

Die Standard-Abfragesprache für **relationale Datenbanken (RDBMS)**.

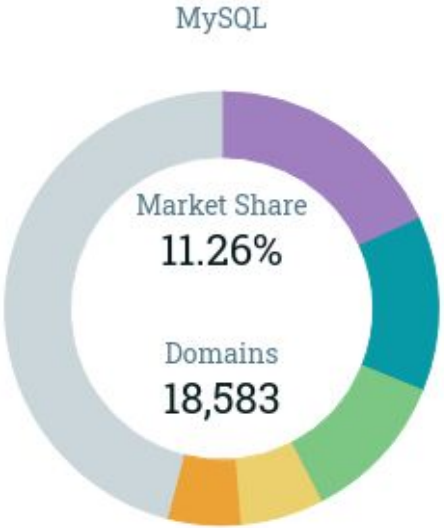
ISO-Standard seit 1987.

Die kostenlosen Systeme MySQL und PostgreSQL sind hinter den meisten kleineren Webseiten und Blogs (z.B. alle Wordpress-Seiten, das sind bereits 40% aller Webseiten).

Top Databases Technologies

Microsoft SQL Server has the top spot for best Databases by market share. Followed by **Microsoft Access** and **MySQL**

1. Microsoft SQL Server
2. Microsoft Access
3. MySQL
4. PostgreSQL
5. Oracle Database (RDBMS)



https://www.datanyze.com/market-share/databases--272

SQL

Heinrich Heine
Universität Düsseldorf

In relationalen Datenbanken sind **Tabellen** (tables) die wichtigsten Objekte.

Ein SQL-Befehl fängt mit einem **Kommando** an:

`SELECT`	Daten auswählen/extrahieren
`UPDATE`	Daten verändern/updaten
`DELETE`	Daten löschen
`INSERT INTO`	Neue Daten hinzufügen
`CREATE DATABASE`	Neue Datenbank
`ALTER DATABASE`	Datenbankparameter verändern
`CREATE TABLE`	Neue Tabelle
`ALTER TABLE`	Tabellenparameter verändern
`DROP TABLE`	Tabelle löschen
`CREATE INDEX`	Index erstellen (Suchschlüssel)
`DROP INDEX`	Index löschen

Beispiel:

```
SELECT "Petal.Width" FROM iris
```

Grundbegriffe relationaler Datenbanken

Heinrich Heine
Universität Düsseldorf

Ein **Datenbankschema** S ist eine endliche, nichtleere Menge von Relationsschemata.

Ein **Relationsschema** ist eine formale Beschreibung der Datentypen einer Relation.

Beispiel einer Relation:

Eine Tabelle heißt bei relationalen Datenbanken auch **Relation**.
Die Zeilen heißen **Entitäten** oder Tupel.
Die Spalten heißen **Attribute**.
Die Werte, die ein Attribut annehmen kann (=die in der Spalte stehen dürfen), also der Datentyp, heißt **Wertebereich** (domain).

Spezies	**Kelchblattbreite**
setosa	3.5
versicolor	3.2
versicolor	3.3

Zwei Datensätze vereinen

Heinrich Heine
Universität Düsseldorf

Wie werden aus diesen zwei eins?

Problem: unklar, ob Zeilen einander entsprechen

Spezies
setosa
versicolor

Kelchblattbreite
3.5
3.2

Zwei Datensätze vereinen

Heinrich Heine
Universität Düsseldorf

Wie werden aus diesen zwei eins?

Lösung: eindeutige Nummerierung (hier: Nr) oder andere eindeutige Information

→ UUID = universally unique identifier
→ Primärschlüssel

Nr	Spezies
1	setosa
51	versicolor

Nr	Kelchblattbreite
1	3.5
51	3.2

Zwei Datensätze

Heinrich Heine
Universität Düsseldorf

x

1	x1
2	x2
3	x3

y

1	y1
2	y2
4	y3

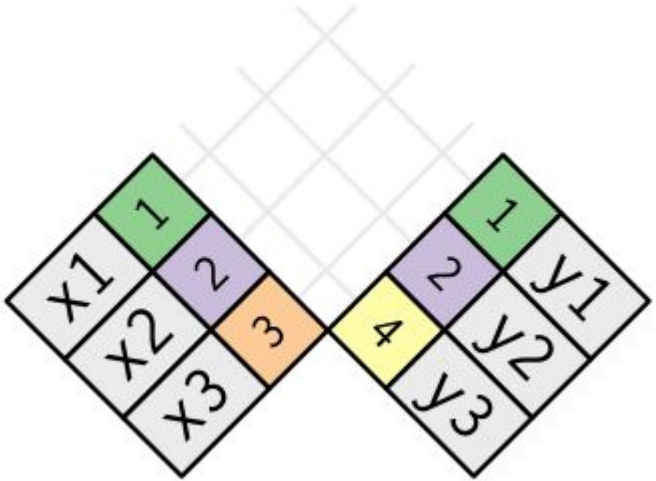
Quelle: R4DS = “R for Data Science”,
https://r4ds.had.co.nz/relational-data.html

Zwei Datensätze

x	
1	x1
2	x2
3	x3

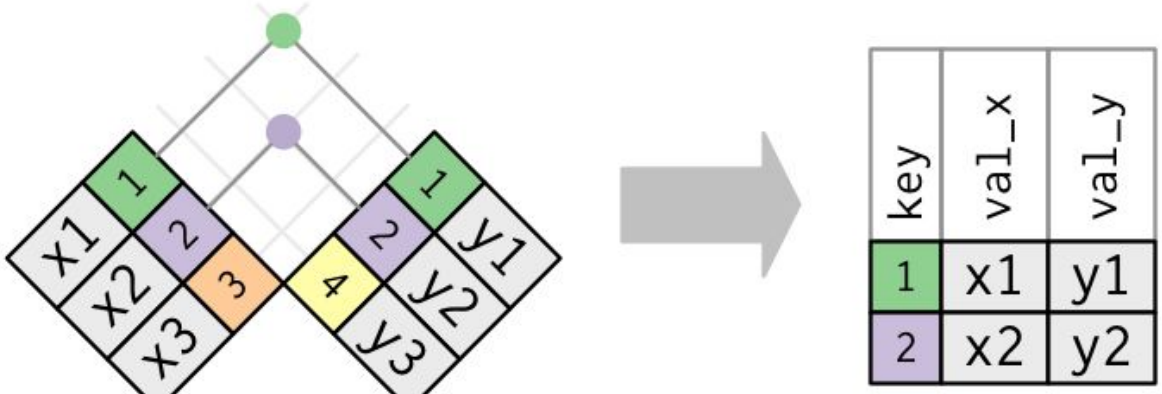
y	
1	y1
2	y2
4	y3

=



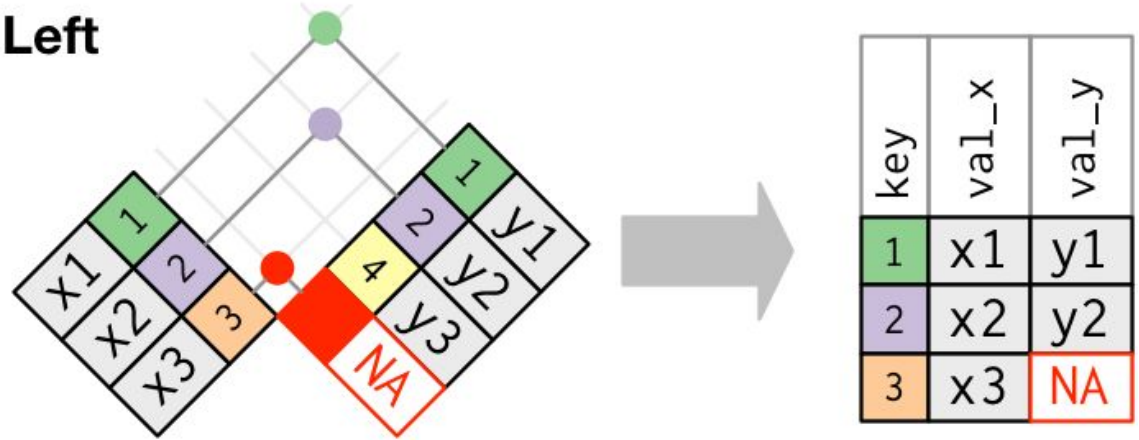
Quelle: R4DS, https://r4ds.had.co.nz/relational-data.html

SELECT * FROM x INNER JOIN y USING (z)



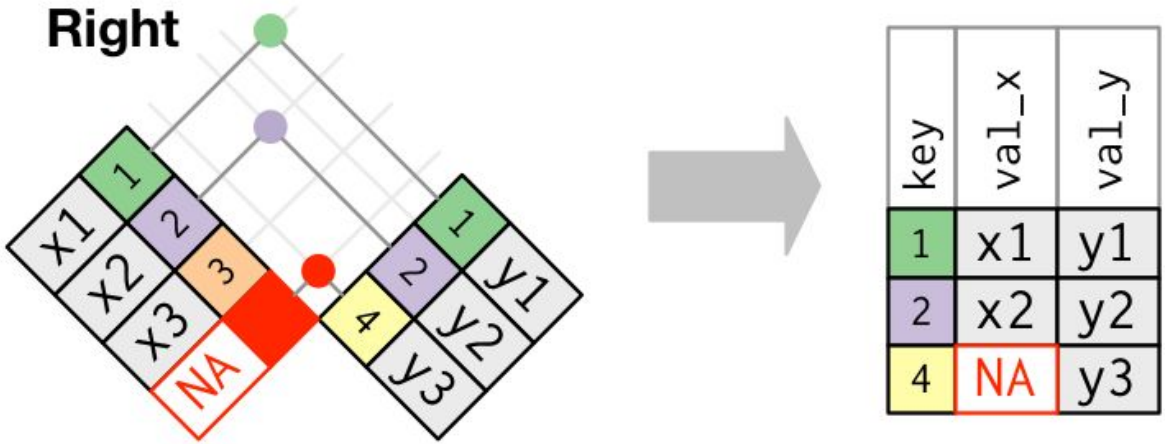
Quelle: R4DS,
https://r4ds.had.co.nz/relational-data.html

SELECT * FROM x LEFT OUTER JOIN y USING (z)



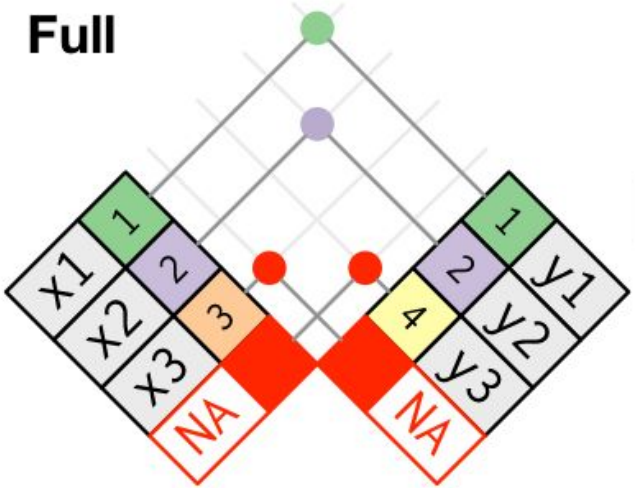
Quelle: R4DS,
https://r4ds.had.co.nz/relational-data.html

SELECT * FROM x RIGHT OUTER JOIN y USING (z)



Quelle: R4DS,
https://r4ds.had.co.nz/relational-data.html

SELECT * FROM x FULL OUTER JOIN y USING (z)



key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA
4	NA	y3

Quelle: R4DS,
https://r4ds.had.co.nz/relational-data.html

SQL → Pandas

Joins & Selects



“Meditating panda, ink”

Setup

Heinrich Heine
Universität Düsseldorf

Aus der Pandas Dokumentation, Abschnitt "Comparison with SQL".

```
import numpy as np
import pandas as pd
url = (
    "https://raw.github.com/pandas-dev"
    "/pandas/main/pandas/tests/io/data/csv/tips.csv"
)
tips = pd.read_csv(url)
print(tips)
```

```
     total_bill   tip     sex smoker   day    time  size
0         16.99  1.01  Female     No   Sun  Dinner     2
1         10.34  1.66    Male     No   Sun  Dinner     3
..          ...   ...     ...    ...   ...     ...   ...
242       17.82  1.75    Male     No   Sat  Dinner     2
243       18.78  3.00  Female     No  Thur  Dinner     2
[244 rows x 7 columns]
```

Pandas gibt zurück

Heinrich Heine
Universität Düsseldorf

Aus der Pandas Dokumentation, Abschnitt "Comparison with SQL".

Operationen auf Pandas DataFrames erzeugen Kopien, während der ursprüngliche DataFrame unverändert bleibt.

```
tips.sort_values("total_bill")
print(tips)
print("\nDas ist ja gar nicht sortiert! Das nächste aber:\n")
sorted_tips = tips.sort_values("total_bill")
print(sorted_tips)
```

```
     total_bill   tip     sex smoker   day    time  size
0         16.99  1.01  Female     No   Sun  Dinner     2
1         10.34  1.66    Male     No   Sun  Dinner     3
2         21.01  3.50    Male     No   Sun  Dinner     3
3         23.68  3.31    Male     No   Sun  Dinner     2
4         24.59  3.61  Female     No   Sun  Dinner     4
..          ...   ...     ...    ...   ...     ...   ...
239       29.03  5.92    Male     No   Sat  Dinner     3
240       27.18  2.00  Female    Yes   Sat  Dinner     2
241       22.67  2.00    Male    Yes   Sat  Dinner     2
242       17.82  1.75    Male     No   Sat  Dinner     2
243       18.78  3.00  Female     No  Thur  Dinner     2

[244 rows x 7 columns]
```

```
Das ist ja gar nicht sortiert! Das nächste aber:

     total_bill    tip     sex smoker   day    time  size
67         3.07   1.00  Female    Yes   Sat  Dinner     1
92         5.75   1.00  Female    Yes   Fri  Dinner     2
111        7.25   1.00  Female     No   Sat  Dinner     1
172        7.25   5.15    Male    Yes   Sun  Dinner     2
149        7.51   2.00    Male     No  Thur   Lunch     2
..          ...    ...     ...    ...   ...     ...   ...
182       45.35   3.50    Male    Yes   Sun  Dinner     3
156       48.17   5.00    Male     No   Sun  Dinner     6
59        48.27   6.73    Male     No   Sat  Dinner     4
212       48.33   9.00    Male     No   Sat  Dinner     4
170       50.81  10.00    Male    Yes   Sat  Dinner     3

[244 rows x 7 columns]
```

```
SELECT total_bill, tip, smoker, time FROM tips;
```

```
tips[["total_bill", "tip", "smoker", "time"]]
```

	total_bill	tip	smoker	time
0	16.99	1.01	No	Dinner
1	10.34	1.66	No	Dinner
2	21.01	3.50	No	Dinner
3	23.68	3.31	No	Dinner
4	24.59	3.61	No	Dinner
...	...	...	...	...
239	29.03	5.92	No	Dinner
240	27.18	2.00	Yes	Dinner
241	22.67	2.00	Yes	Dinner
242	17.82	1.75	No	Dinner
243	18.78	3.00	No	Dinner

244 rows × 4 columns

```
SELECT *, tip/total_bill as tip_rate FROM tips;
```

```
tips.assign(tip_rate=tips["tip"] / tips["total_bill"])
```

	total_bill	tip	sex	smoker	day	time	size	tip_rate
0	16.99	1.01	Female	No	Sun	Dinner	2	0.059447
1	10.34	1.66	Male	No	Sun	Dinner	3	0.160542
2	21.01	3.50	Male	No	Sun	Dinner	3	0.166587
3	23.68	3.31	Male	No	Sun	Dinner	2	0.139780
4	24.59	3.61	Female	No	Sun	Dinner	4	0.146808
...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	0.203927
240	27.18	2.00	Female	Yes	Sat	Dinner	2	0.073584
241	22.67	2.00	Male	Yes	Sat	Dinner	2	0.088222
242	17.82	1.75	Male	No	Sat	Dinner	2	0.098204
243	18.78	3.00	Female	No	Thur	Dinner	2	0.159744

244 rows × 8 columns

```
SELECT * FROM tips WHERE time = 'Dinner';
```

```
tips[tips["time"] == "Dinner"]
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

176 rows × 7 columns

```
SELECT * FROM tips WHERE time = 'Dinner' AND tip > 6.00;
```

Heinrich Heine
Universität Düsseldorf

```
tips[(tips["time"] == "Dinner") & (tips["tip"] > 6.00)]
```

	total_bill	tip	sex	smoker	day	time	size
23	39.42	7.58	Male	No	Sat	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
170	50.81	10.00	Male	Yes	Sat	Dinner	3
183	23.17	6.50	Male	Yes	Sun	Dinner	4
212	48.33	9.00	Male	No	Sat	Dinner	4
214	28.17	6.50	Female	Yes	Sat	Dinner	3

```
SELECT * FROM tips WHERE size >= 6 OR total_bill > 45;
```

```
tips[(tips["size"] >= 6) | (tips["total_bill"] > 45)]
```

	total_bill	tip	sex	smoker	day	time	size
59	48.27	6.73	Male	No	Sat	Dinner	4
125	29.80	4.20	Female	No	Thur	Lunch	6
141	34.30	6.70	Male	No	Thur	Lunch	6
143	27.05	5.00	Female	No	Thur	Lunch	6
156	48.17	5.00	Male	No	Sun	Dinner	6
170	50.81	10.00	Male	Yes	Sat	Dinner	3
182	45.35	3.50	Male	Yes	Sun	Dinner	3
212	48.33	9.00	Male	No	Sat	Dinner	4

```
SELECT sex, count(*) FROM tips GROUP BY sex;
```

```
tips.groupby("sex").size()
```

```
sex
Female     87
Male      157
dtype: int64
```

```
SELECT day, AVG(tip), COUNT(*) FROM tips GROUP BY day;
```

Heinrich Heine Universität Düsseldorf

```
tips.groupby("day").agg({"tip": np.mean, "day": np.size})
```

day	tip	day
Fri	2.734737	19
Sat	2.993103	87
Sun	3.255132	76
Thur	2.771452	62

```
SELECT smoker, day, COUNT(*), AVG(tip) FROM tips GROUP BY smoker, day;
```

Heinrich Heine Universität Düsseldorf

```
tips.groupby(["smoker", "day"]).agg({"tip": [np.size, np.mean]})
```

		tip	
		size	mean
smoker	day		
No	Fri	4.0	2.812500
	Sat	45.0	3.102889
	Sun	57.0	3.167895
	Thur	45.0	2.673778
Yes	Fri	15.0	2.714000
	Sat	42.0	2.875476
	Sun	19.0	3.516842
	Thur	17.0	3.030000

Setup vor dem JOIN

```
df1 = pd.DataFrame({"key": ["A", "B", "C", "D"], "value": np.random.randn(4)})
df2 = pd.DataFrame({"key": ["B", "D", "D", "E"], "value": np.random.randn(4)})
print(df1, "\n", df2)
```

```
 key     value
0   A -0.093612
1   B  0.664115
2   C -0.167359
3   D -0.105395
   key     value
0   B -1.108029
1   D  1.150343
2   D  0.714288
3   E -0.190992
```

```
SELECT * FROM df1 INNER JOIN df2 ON df1.key = df2.key;
```

```
pd.merge(df1, df2, on="key") # inner join ist der default
```

	key	value_x	value_y
0	B	0.664115	-1.108029
1	D	-0.105395	1.150343
2	D	-0.105395	0.714288

```
SELECT * FROM df1 LEFT OUTER JOIN df2 ON df1.key = df2.key;
```

```
pd.merge(df1, df2, on="key", how="left")
```

	key	value_x	value_y
0	A	-0.093612	NaN
1	B	0.664115	-1.108029
2	C	-0.167359	NaN
3	D	-0.105395	1.150343
4	D	-0.105395	0.714288

```
SELECT * FROM df1 RIGHT OUTER JOIN df2 ON df1.key = df2.key;
```

```
pd.merge(df1, df2, on="key", how="right")
```

	key	value_x	value_y
0	B	0.664115	-1.108029
1	D	-0.105395	1.150343
2	D	-0.105395	0.714288
3	E	NaN	-0.190992

```
SELECT * FROM df1 FULL OUTER JOIN df2 ON df1.key = df2.key;
```

```
pd.merge(df1, df2, on="key", how="outer")
```

	key	value_x	value_y
0	A	-0.093612	NaN
1	B	0.664115	-1.108029
2	C	-0.167359	NaN
3	D	-0.105395	1.150343
4	D	-0.105395	0.714288
5	E	NaN	-0.190992

Zustand vor dem Ändern der Daten:

Heinrich Heine
Universität Düsseldorf

```
tips.sort_values("tip").head()
```

	total_bill	tip	sex	smoker	day	time	size
67	3.07	1.00	Female	Yes	Sat	Dinner	1
236	12.60	1.00	Male	Yes	Sat	Dinner	2
92	5.75	1.00	Female	Yes	Fri	Dinner	2
111	7.25	1.00	Female	No	Sat	Dinner	1
0	16.99	1.01	Female	No	Sun	Dinner	2

```
UPDATE tips SET tip = tip*2 WHERE tip < 2;
```

```
tips.loc[tips["tip"] < 2, "tip"] *= 2
tips.sort_values("tip").head()
```

	total_bill	tip	sex	smoker	day	time	size
67	3.07	2.0	Female	Yes	Sat	Dinner	1
159	16.49	2.0	Male	No	Sun	Dinner	4
162	16.21	2.0	Female	No	Sun	Dinner	3
61	13.81	2.0	Male	Yes	Sat	Dinner	2
163	13.81	2.0	Male	No	Sun	Dinner	2

Zustand vor der nächsten Änderung

Heinrich Heine
Universität Düsseldorf

```
tips.sort_values("tip", ascending=False).head()
```

	total_bill	tip	sex	smoker	day	time	size
170	50.81	10.00	Male	Yes	Sat	Dinner	3
212	48.33	9.00	Male	No	Sat	Dinner	4
23	39.42	7.58	Male	No	Sat	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
141	34.30	6.70	Male	No	Thur	Lunch	6

```
DELETE FROM tips WHERE tip > 9;
```

```
tips = tips.loc[tips["tip"] <= 9]
tips.sort_values("tip", ascending=False).head()
```

	total_bill	tip	sex	smoker	day	time	size
212	48.33	9.00	Male	No	Sat	Dinner	4
23	39.42	7.58	Male	No	Sat	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
141	34.30	6.70	Male	No	Thur	Lunch	6
183	23.17	6.50	Male	Yes	Sun	Dinner	4

Danke für Ihre Aufmerksamkeit.